МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №4

з курсу: „ Інтернет речей та вбудовані системи”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

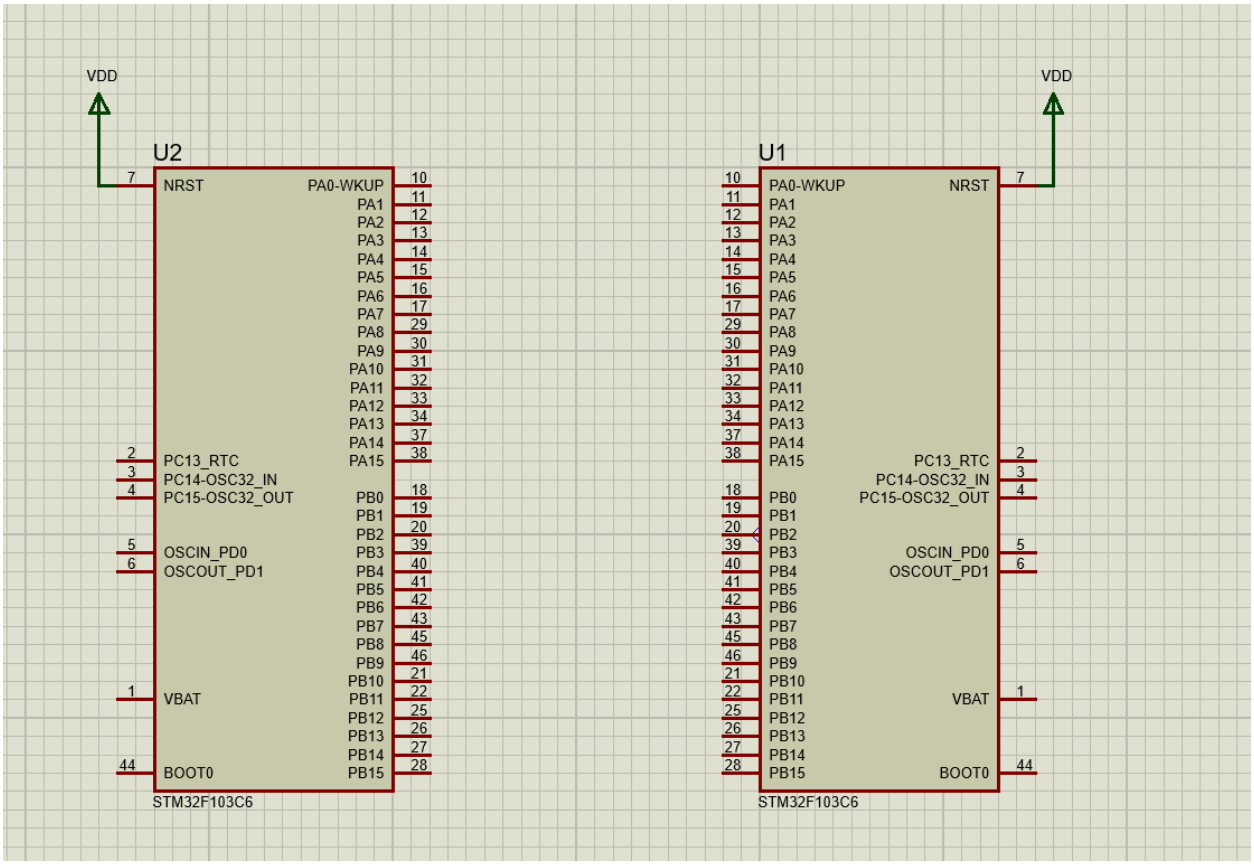
Мета роботи: реалізувати в САПР Proteus послідовні інтерфейси та вивчити принцип їх роботи.

Варіант 21

Хід роботи

1.1 Створити проект, що складено з двох МК STM32F103C6.

Створимо проект, що буде складатися з двох мікроконтролерів МК STM32F103C6.



1.2. Написати програму, що реалізує обмін даними між двома МК за перериваннями у повнодуплексному режимі. Навести діаграми обміну даними між МК.

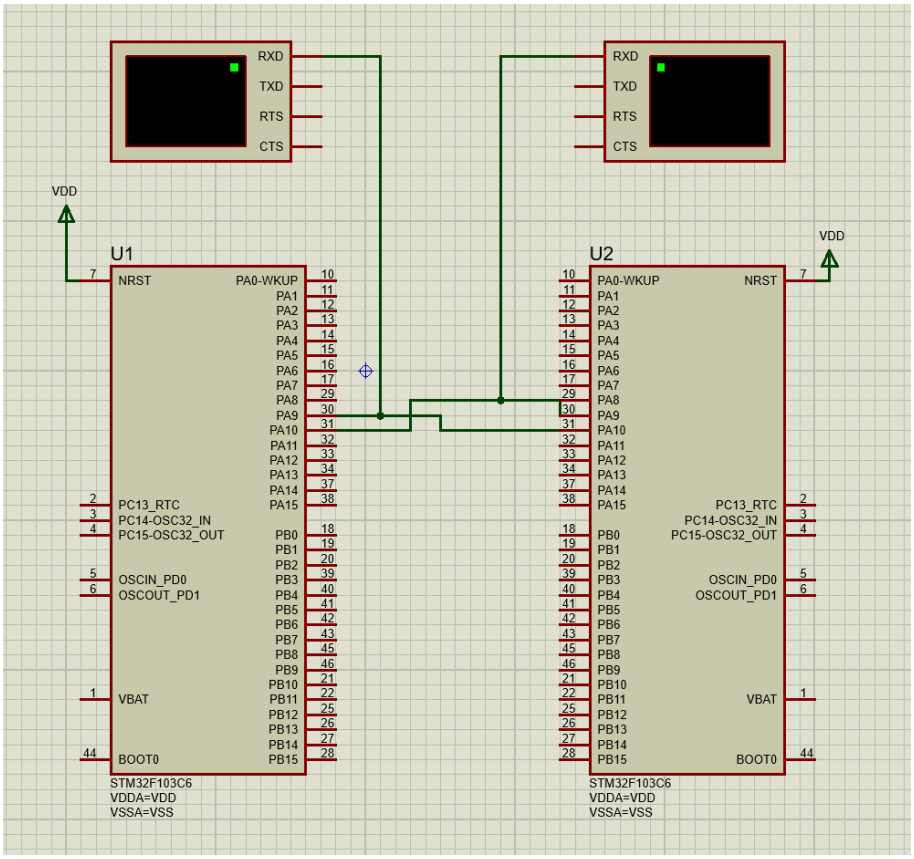
Таблиця 1

Вар.	Швидкість, Бод/с	Довжина слова, біт	Кількість стоп біт	Перевірка парності	Номер UART (1 МК / 2МК)
1	9600	5	0.5	+	1/1

Для виконання пункту було реалізовано обмін даними між двома мікроконтролерами STM32F103C6 з використанням послідовного інтерфейсу UART у повнодуплексному режимі. Налаштування параметрів інтерфейсу виконувалося відповідно до індивідуального варіанта, наведеного у таблиці 1.

Для обох мікроконтролерів було використано апаратний модуль USART1 з однаковими параметрами швидкості передавання, формату кадру та перевірки парності. Обмін даними організовано з використанням механізму апаратних переривань, що дозволяє уникнути постійного опитування стану інтерфейсу в основному циклі програми.

На рисунку наведено схему з’єднання двох мікроконтролерів STM32F103C6 у середовищі Proteus для реалізації повнодуплексного обміну даними. Передавач TX першого мікроконтролера підключено до приймача RX другого, а передавач TX другого мікроконтролера з’єднано з приймачем RX першого. Таке перехресне з’єднання є стандартним для організації UART-зв’язку між двома пристроями. Для забезпечення коректної роботи інтерфейсу також об’єднано загальну шину землі GND обох мікроконтролерів. Використання окремих ліній TX та RX дозволяє здійснювати одночасну передачу і прийом даних, що відповідає повнодуплексному режиму роботи.



Source Code

*main.c

```c
#include "stm32f1xx.h"

/*
 * Обробник переривання USART1.
 * Викликається при прийомі нового байта даних.
 */
void USART1_IRQHandler(void)
{
    if (USART1->SR & USART_SR_RXNE)
    {
        /* Зчитування прийнятого значення (5 інформаційних бітів) */
        uint8_t rx = USART1->DR & 0x1F;   // діапазон 0…31

        /* Обмеження значення до 0…9 */
        rx = rx % 10;

        /* Формування відповіді */
        uint8_t tx = (rx + 1) % 10;

        /* Передача відповіді у форматі ASCII */
        USART1->DR = tx + '0';
    }
}

int main(void)
{
    /* Увімкнення тактування GPIOA, AFIO та USART1 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
                    RCC_APB2ENR_AFIOEN |
                    RCC_APB2ENR_USART1EN;

    /* Налаштування PA9 як TX USART1 (AF Push-Pull) */
    GPIOA->CRH &= ~GPIO_CRH_CNF9;
    GPIOA->CRH |=  GPIO_CRH_CNF9_1;
    GPIOA->CRH |=  GPIO_CRH_MODE9;

    /* Налаштування PA10 як RX USART1 (вхід з плаваючим рівнем) */
    GPIOA->CRH &= ~GPIO_CRH_CNF10;
    GPIOA->CRH |=  GPIO_CRH_CNF10_0;
    GPIOA->CRH &= ~GPIO_CRH_MODE10;

    /* Встановлення швидкості обміну 9600 бод при 8 МГц */
    USART1->BRR = 0x341;

    /* Формат кадру: 8 біт даних, без парності */
    USART1->CR1 &= ~USART_CR1_M;
    USART1->CR1 &= ~USART_CR1_PCE;

    /* Встановлення 0.5 стоп-біта */
    USART1->CR2 &= ~USART_CR2_STOP;
    USART1->CR2 |= USART_CR2_STOP_0;

    /* Увімкнення передавача, приймача, переривань та USART */
    USART1->CR1 |= USART_CR1_RE |
                   USART_CR1_TE |
                   USART_CR1_RXNEIE |
                   USART_CR1_UE;

    /* Дозвіл переривань USART1 */
    NVIC_EnableIRQ(USART1_IRQn);
    __enable_irq();

    /* Передача стартового байта для ініціації обміну */
    while (!(USART1->SR & USART_SR_TXE));
    USART1->DR = 0x01;

    /* Основний цикл */
    while (1) {}
}
```

*main.c

```c
#include "stm32f1xx.h"

/*
 * Обробник переривання USART1.
 * Викликається при прийомі байта від першого МК.
 */
void USART1_IRQHandler(void)
{
    if (USART1->SR & USART_SR_RXNE)
    {
        /* Зчитування прийнятого значення */
        uint8_t rx = USART1->DR & 0x1F;

        /* Обмеження до діапазону 0…9 */
        rx = rx % 10;

        /* Формування відповіді */
        uint8_t tx = (rx + 1) % 10;

        /* Передача відповіді у форматі ASCII */
        USART1->DR = tx + '0';
    }
}

int main(void)
{
    /* Увімкнення тактування GPIOA, AFIO та USART1 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
                    RCC_APB2ENR_AFIOEN |
                    RCC_APB2ENR_USART1EN;

    /* PA9 – TX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF9;
    GPIOA->CRH |=  GPIO_CRH_CNF9_1;
    GPIOA->CRH |=  GPIO_CRH_MODE9;

    /* PA10 – RX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF10;
    GPIOA->CRH |=  GPIO_CRH_CNF10_0;
    GPIOA->CRH &= ~GPIO_CRH_MODE10;

    /* Швидкість обміну 9600 бод */
    USART1->BRR = 0x341;

    /* 8 біт даних, без перевірки парності */
    USART1->CR1 &= ~USART_CR1_M;
    USART1->CR1 &= ~USART_CR1_PCE;

    /* 0.5 стоп-біта */
    USART1->CR2 &= ~USART_CR2_STOP;
    USART1->CR2 |= USART_CR2_STOP_0;

    /* Увімкнення USART, RX, TX та переривань */
    USART1->CR1 |= USART_CR1_RE |
                   USART_CR1_TE |
                   USART_CR1_RXNEIE |
                   USART_CR1_UE;

    NVIC_EnableIRQ(USART1_IRQn);
    __enable_irq();

    /* Основний цикл */
    while (1) {}
}
```

Для реалізації обміну даними між двома мікроконтролерами STM32F103C6 було розроблено дві окремі програми, які виконуються на кожному з мікроконтролерів. Обмін здійснюється за допомогою апаратного інтерфейсу USART1 у повнодуплексному режимі з використанням механізму апаратних переривань. Один мікроконтролер виконує роль ініціатора обміну, а другий мікроконтролер формує відповідь на отримані дані.

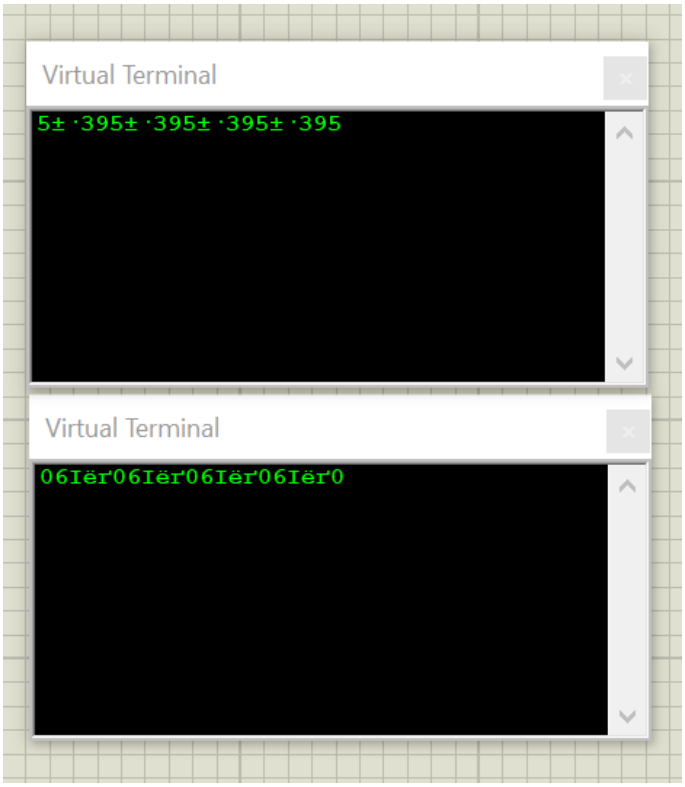
В обох програмах виконується ініціалізація тактування порту GPIOA, модуля альтернативних функцій та модуля USART1. Вивід PA9 налаштовано як лінію передавання даних TX, а вивід PA10 як лінію приймання даних RX. Таке налаштування забезпечує коректну роботу апаратного UART відповідно до стандартної схеми підключення. Швидкість передавання даних встановлено рівною 9600 бод при тактовій частоті мікроконтролера 8 МГц.

Формат кадру UART налаштовано на передачу восьми бітів даних без перевірки парності з використанням половини стоп-біта. Прийом даних реалізовано за допомогою переривання RXNE. При надходженні нового байта

викликається обробник переривання USART1_IRQHandler, в якому відбувається зчитування прийнятого значення з регістра даних USART та формування відповіді, яка одразу передається іншому мікроконтролеру.

Перший мікроконтролер додатково передає стартовий байт, що ініціює процес обміну даними. Подальший обмін здійснюється автоматично за рахунок взаємного виклику обробників переривань на обох мікроконтролерах. Основний цикл програм не містить логіки обміну даними, що підтверджує реалізацію обміну за перериваннями у повнодуплексному режимі.

Тоді маємо такий результат роботи коду:

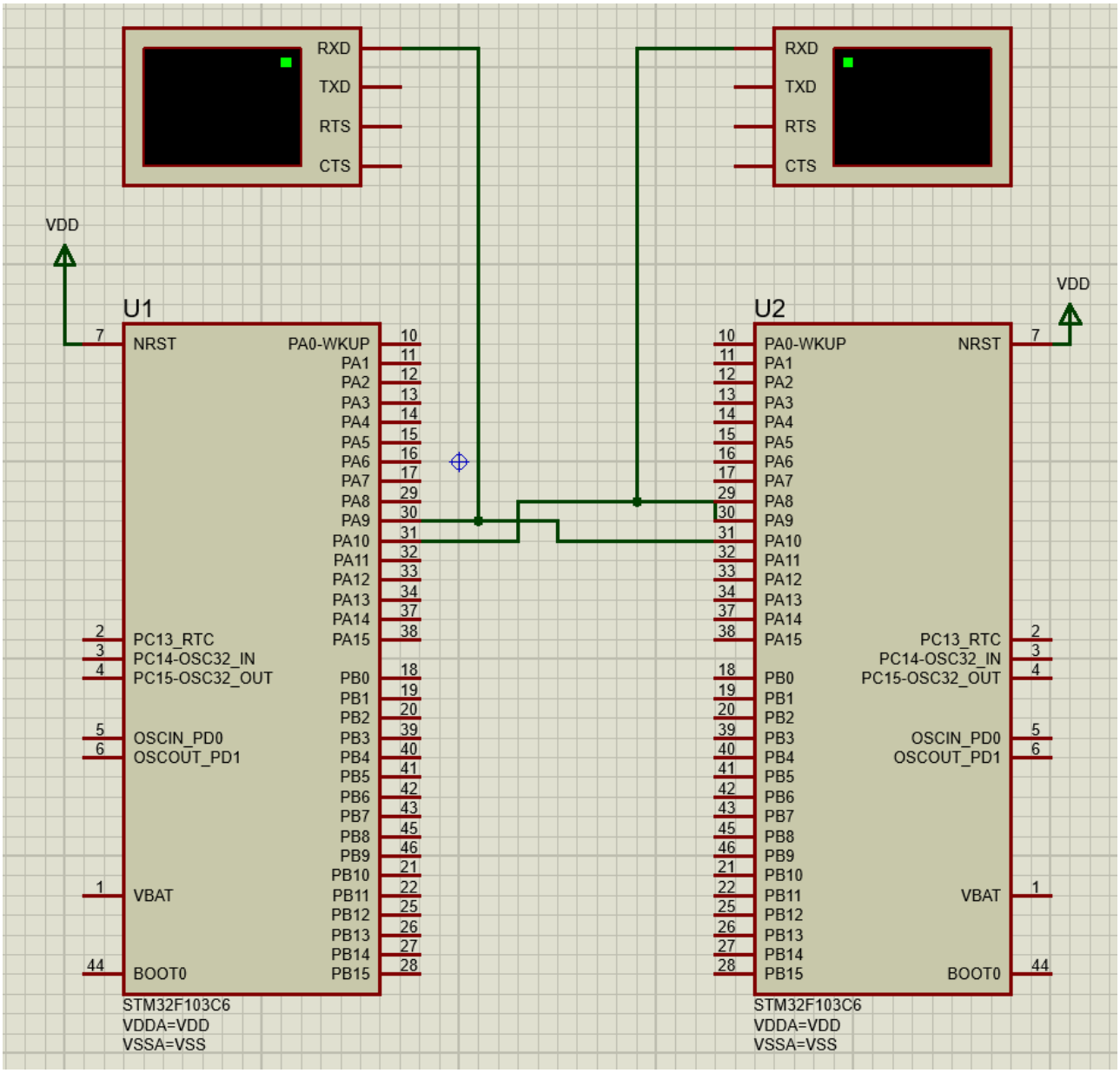


1.3. Переробити проект та код програми для обміну даними у напів дуплексному режимі з тими ж параметрами. Навести діаграми обміну даними між МК.

На відміну від повнодуплексного режиму, у якому передавання та приймання даних можуть відбуватися одночасно, у напівдуплексному режимі обмін здійснюється послідовно, тобто в кожен момент часу мікроконтролер або передає дані, або приймає їх. Параметри інтерфейсу UART залишилися незмінними та відповідають значенням, заданим у таблиці варіанта.

Для реалізації напівдуплексного обміну було використано той самий апаратний інтерфейс USART1, що і в попередньому пункті. Зміни стосувалися лише програмної логіки обміну. Було розроблено дві окремі програми для кожного мікроконтролера. Перший мікроконтролер виконує роль ініціатора обміну та формує запит, а другий мікроконтролер виконує роль відповідача та надсилає відповідь після прийому даних.

На рисунку наведено схему з’єднання двох мікроконтролерів STM32F103C6 у середовищі Proteus для реалізації напівдуплексного обміну даними. Виводи PA9 та PA10 використовуються відповідно як лінії передавання та приймання даних інтерфейсу USART1. Обидва мікроконтролери мають спільну землю, що забезпечує коректну роботу цифрового інтерфейсу. Фізичне з’єднання мікроконтролерів не відрізняється від схеми, використаної для повнодуплексного режиму, однак режим напівдуплексу забезпечується програмно шляхом керування послідовністю операцій передавання та приймання.



Source Code

*main.c

```c
#include "stm32f1xx.h"

/* Проста програмна затримка */
void delay(volatile uint32_t count)
{
    while (count--);
}

int main(void)
{
    /* Увімкнення тактування GPIOA, AFIO та USART1 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
                    RCC_APB2ENR_AFIOEN |
                    RCC_APB2ENR_USART1EN;

    /* Налаштування PA9 як лінії TX USART1 (AF Push-Pull) */
    GPIOA->CRH &= ~GPIO_CRH_CNF9;
    GPIOA->CRH |=  GPIO_CRH_CNF9_1;
    GPIOA->CRH |=  GPIO_CRH_MODE9;

    /* Налаштування PA10 як лінії RX USART1 (вхід з плаваючим рівнем) */
    GPIOA->CRH &= ~GPIO_CRH_CNF10;
    GPIOA->CRH |=  GPIO_CRH_CNF10_0;
    GPIOA->CRH &= ~GPIO_CRH_MODE10;

    /* Встановлення швидкості обміну 9600 бод при 8 МГц */
    USART1->BRR = 0x341;

    /* Формат кадру: 8 біт даних з перевіркою парності EVEN */
    USART1->CR1 &= ~USART_CR1_M;
    USART1->CR1 |= USART_CR1_PCE;
    USART1->CR1 &= ~USART_CR1_PS;

    /* Встановлення одного стоп-біта */
    USART1->CR2 &= ~USART_CR2_STOP;

    /* Увімкнення передавача, приймача та USART */
    USART1->CR1 |= USART_CR1_RE |
                   USART_CR1_TE |
                   USART_CR1_UE;

    uint8_t char_to_send = '0';

    while (1)
    {
        /* Передача символу */
        while (!(USART1->SR & USART_SR_TXE));
        USART1->DR = char_to_send;

        /* Очікування завершення передачі */
        while (!(USART1->SR & USART_SR_TC));

        /* Очікування відповіді */
        while (!(USART1->SR & USART_SR_RXNE));
        uint8_t received_char = USART1->DR;

        /* Обробка отриманих даних */
        uint8_t next_val = ((received_char - '0') + 1) % 10;
        char_to_send = next_val + '0';

        /* Затримка між циклами обміну */
        delay(500000);
    }
}
```

Source Code

*main.c

```c
#include "stm32f1xx.h"

int main(void)
{
    /* Увімкнення тактування GPIOA, AFIO та USART1 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
                    RCC_APB2ENR_AFIOEN |
                    RCC_APB2ENR_USART1EN;

    /* Налаштування PA9 як TX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF9;
    GPIOA->CRH |=  GPIO_CRH_CNF9_1;
    GPIOA->CRH |=  GPIO_CRH_MODE9;

    /* Налаштування PA10 як RX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF10;
    GPIOA->CRH |=  GPIO_CRH_CNF10_0;
    GPIOA->CRH &= ~GPIO_CRH_MODE10;

    /* Швидкість обміну 9600 бод */
    USART1->BRR = 0x341;

    /* Формат кадру: 8 біт даних з перевіркою парності EVEN */
    USART1->CR1 &= ~USART_CR1_M;
    USART1->CR1 |= USART_CR1_PCE;
    USART1->CR1 &= ~USART_CR1_PS;

    /* Один стоп-біт */
    USART1->CR2 &= ~USART_CR2_STOP;

    /* Увімкнення передавача, приймача та USART */
    USART1->CR1 |= USART_CR1_RE |
                   USART_CR1_TE |
                   USART_CR1_UE;

    while (1)
    {
        /* Очікування прийому даних */
        while (!(USART1->SR & USART_SR_RXNE));
        uint8_t received_char = USART1->DR;

        /* Обробка отриманого символу */
        uint8_t next_val = ((received_char - '0') + 5) % 10;
        uint8_t char_to_send = next_val + '0';

        /* Передача відповіді */
        while (!(USART1->SR & USART_SR_TXE));
        USART1->DR = char_to_send;

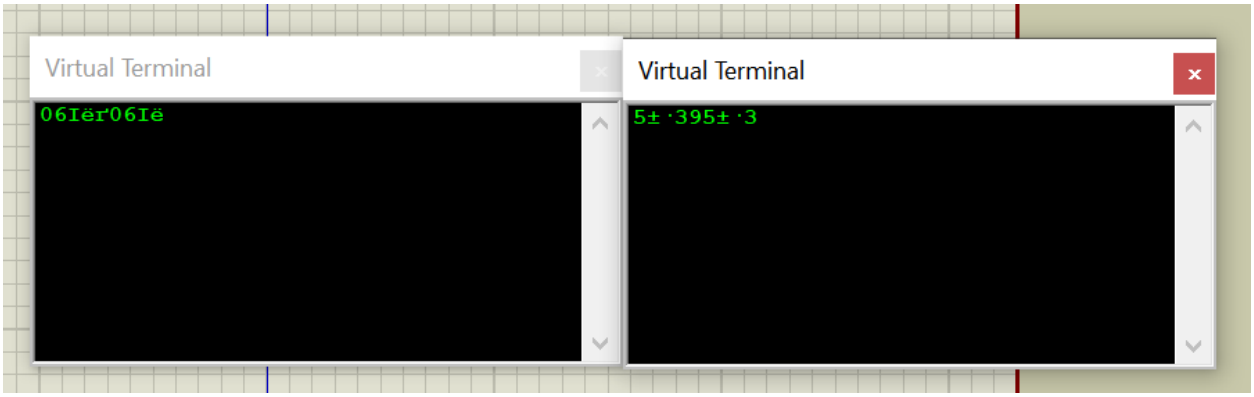
        /* Очікування завершення передачі */
        while (!(USART1->SR & USART_SR_TC));
    }
}
```

Для реалізації напівдуплексного обміну було розроблено дві окремі програми. Перший мікроконтролер виконує роль ініціатора обміну і періодично передає символи іншому мікроконтролеру. Після завершення передачі він переходить у режим очікування відповіді та зчитує прийняті дані. Другий мікроконтролер постійно перебуває в режимі очікування прийому символу, після чого виконує обробку отриманих даних і формує відповідь, яка передається назад першому мікроконтролеру.

В обох програмах здійснюється ініціалізація тактування порту GPIOA, модуля альтернативних функцій та модуля USART1. Виводи PA9 і PA10 використовуються відповідно як лінії передавання та приймання даних. Швидкість передавання встановлено рівною 9600 бод при тактовій частоті мікроконтролера 8 МГц. Формат кадру налаштовано на передачу восьми бітів даних з використанням перевірки парності EVEN та одного стоп-біта.

Обмін даними реалізовано шляхом програмного керування чергуванням операцій передавання та приймання. Перший мікроконтролер передає символ, очікує завершення передачі, після чого переходить у режим прийому. Другий мікроконтролер після прийому символу формує відповідь та передає її у

зворотному напрямку. Така організація обміну забезпечує коректну роботу системи у напівдуплексному режимі. Отримані часові діаграми сигналів підтверджують, що передавання і приймання даних не відбуваються одночасно.



Таким чином, обмін даними між мікроконтролерами здійснюється почергово, що відповідає напівдуплексному режиму роботи. Передавання та приймання даних не відбуваються одночасно, а чітко розділені у часі. Результати роботи програми підтверджуються часовими діаграмами сигналів, отриманими за допомогою цифрового осцилографа в середовищі Proteus. На діаграмах спостерігається почергова активність ліній передавання та приймання, що підтверджує коректну реалізацію напівдуплексного обміну даними між двома мікроконтролерами STM32F103C6.

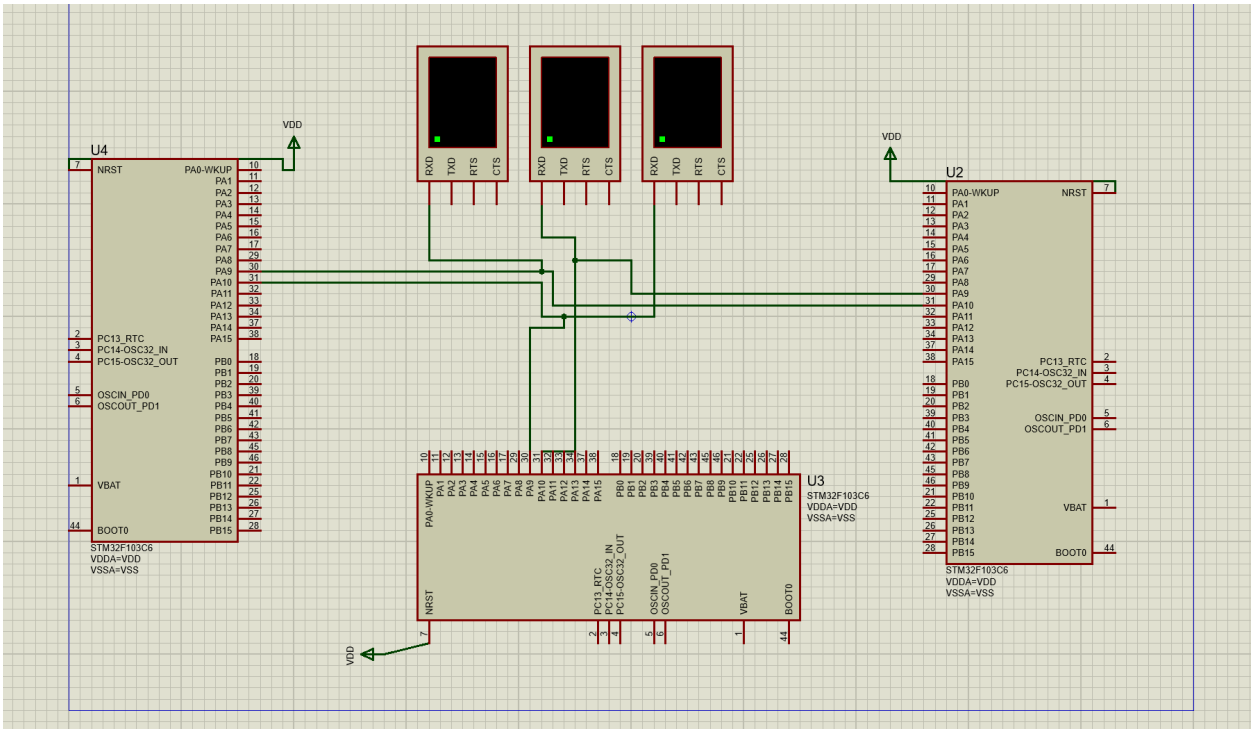
1.4. Під’єднати до системи третій МК і створити програму для обміну даними між МК по будь-якому одноранговому інтерфейсу.

Реалізація обміну даними між декількома мікроконтролерами по одноранговому інтерфейсу, у якому всі вузли мають рівні права та можуть виконувати як передавання, так і приймання даних без виділення ведучого пристрою. Для організації обміну було використано апаратний інтерфейс USART1.

Для реалізації однорангового обміну було використано три мікроконтролери STM32F103C6, з’єднані між собою за допомогою ліній передавання та приймання інтерфейсу UART. Виводи PA9 кожного мікроконтролера використовуються як лінії передавання даних TX, а виводи PA10 як лінії приймання даних RX. Усі мікроконтролери мають спільну шину землі, що забезпечує коректну роботу цифрового інтерфейсу. Така схема з’єднання дозволяє організувати прямий обмін даними між усіма вузлами системи.

На рисунку наведено схему підключення трьох мікроконтролерів у середовищі Proteus. До кожного мікроконтролера додатково підключено

віртуальний термінал, що дозволяє спостерігати результати обміну даними у текстовому вигляді. Обмін інформацією між вузлами здійснюється через спільний канал UART, а всі мікроконтролери працюють у режимі приймання та передавання даних з використанням апаратних переривань.



Для реалізації обміну даними було розроблено три окремі програми, по одній для кожного мікроконтролера. У кожній програмі виконується ініціалізація тактування порту GPIOA, модуля альтернативних функцій та модуля USART1. Швидкість передавання даних встановлено рівною 9600 бод при тактовій частоті мікроконтролера 8 МГц. Формат кадру UART налаштовано на передачу восьми бітів даних без перевірки парності з використанням одного стоп біта. Прийом даних у кожному мікроконтролері реалізовано за допомогою апаратного переривання RXNE.

Алгоритм обміну даними побудований таким чином, що один з мікроконтролерів ініціює передавання початкового символу. Після прийому символу кожен мікроконтролер виконує програмну обробку отриманих даних шляхом збільшення числового значення на одиницю та передає результат назад у канал зв’язку. Таким чином формується послідовний одноранговий обмін даними між всіма трьома мікроконтролерами без використання централізованого керування.

Основний цикл програм усіх мікроконтролерів не містить логіки обміну даними та залишається порожнім. Усі операції приймання, обробки та передавання інформації виконуються в обробнику переривання USART1_IRQHandler. Такий підхід дозволяє ефективно використовувати ресурси мікроконтролерів та забезпечує коректну реалізацію однорангового обміну даними.

Частина 1

```c
#include "stm32f1xx.h"

/*
 * Обробник переривання USART1.
 * Викликається при надходженні байта даних по інтерфейсу UART.
 */
void USART1_IRQHandler(void)
{
    /* Перевірка прапора прийому даних */
    if (USART1->SR & USART_SR_RXNE)
    {
        /* Зчитування прийнятого символу */
        uint8_t received_char = USART1->DR;

        /* Обробка даних: збільшення значення на 1 */
        uint8_t next_val = ((received_char - '0') + 1) % 10;

        /* Очікування готовності передавача */
        while (!(USART1->SR & USART_SR_TXE));

        /* Передача обробленого символу у зворотному напрямку */
        USART1->DR = next_val + '0';
    }
}

int main(void)
{
    /* Увімкнення тактування порту GPIOA, модуля AFIO та USART1 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
        RCC_APB2ENR_AFIOEN |
        RCC_APB2ENR_USART1EN;

    /* Налаштування PA9 як TX USART1 (альтернативна функція, Push-Pull) */
    GPIOA->CRH &= ~GPIO_CRH_CNF9;
    GPIOA->CRH |= GPIO_CRH_CNF9_1;
    GPIOA->CRH |= GPIO_CRH_MODE9;

    /* Налаштування PA10 як RX USART1 (вхід з плаваючим рівнем) */
    GPIOA->CRH &= ~GPIO_CRH_CNF10;
    GPIOA->CRH |= GPIO_CRH_CNF10_0;
    GPIOA->CRH &= ~GPIO_CRH_MODE10;

    /* Налаштування USART1: швидкість 9600 бод при частоті 8 МГц */
    USART1->BRR = 0x341;

    /* Формат кадру: 8 біт даних, без перевірки парності */
    USART1->CR1 &= ~USART_CR1_M;
    USART1->CR1 &= ~USART_CR1_PCE;

    /* Один стоп-біт */
    USART1->CR2 &= ~USART_CR2_STOP;

    /* Увімкнення приймача, передавача, переривання RX та самого USART */
    USART1->CR1 |= USART_CR1_RE |
        USART_CR1_TE |
        USART_CR1_RXNEIE |
        USART_CR1_UE;

    /* Дозвіл переривань USART1 у NVIC */
    NVIC_EnableIRQ(USART1_IRQn);
    __enable_irq();
```

```
    /* Передача початкового символу для запуску обміну */
    while (!(USART1->SR & USART_SR_TXE));
    USART1->DR = '0';

    /* Основний цикл програми */
    while (1)
    {
        /* Обмін даними виконується виключно у перериваннях */
    }
}
```

Частина 2

```
#include "stm32f1xx.h"

/*
 * Обробник переривання USART1.
 * Викликається при прийомі символу від іншого мікроконтролера.
 */
void USART1_IRQHandler(void)
{
    if (USART1->SR & USART_SR_RXNE)
    {
        /* Зчитування прийнятого символу */
        uint8_t received_char = USART1->DR;

        /* Обробка даних: збільшення значення на 1 */
        uint8_t next_val = ((received_char - '0') + 1) % 10;

        /* Очікування готовності передавача */
        while (!(USART1->SR & USART_SR_TXE));

        /* Передача відповіді */
        USART1->DR = next_val + '0';
    }
}

int main(void)
{
    /* Увімкнення тактування GPIOA, AFIO та USART1 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
        RCC_APB2ENR_AFIOEN |
        RCC_APB2ENR_USART1EN;

    /* PA9 – TX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF9;
    GPIOA->CRH |= GPIO_CRH_CNF9_1;
    GPIOA->CRH |= GPIO_CRH_MODE9;

    /* PA10 – RX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF10;
    GPIOA->CRH |= GPIO_CRH_CNF10_0;
    GPIOA->CRH &= ~GPIO_CRH_MODE10;

    /* Швидкість обміну 9600 бод */
    USART1->BRR = 0x341;

    /* 8 біт даних, без перевірки парності */
    USART1->CR1 &= ~USART_CR1_M;
    USART1->CR1 &= ~USART_CR1_PCE;

    /* Один стоп-біт */
```

```
    USART1->CR2 &= ~USART_CR2_STOP;

    /* Увімкнення USART, RX, TX та переривань */
    USART1->CR1 |= USART_CR1_RE |
        USART_CR1_TE |
        USART_CR1_RXNEIE |
        USART_CR1_UE;

    /* Дозвіл переривань USART1 */
    NVIC_EnableIRQ(USART1_IRQn);
    __enable_irq();

    /* Основний цикл */
    while (1)
    {
        /* Взаємодія відбувається у перериваннях */
    }
}
```

Частина 3

```
#include "stm32f1xx.h"
/*
 * Обробник переривання USART1.
 * Забезпечує прийом та передачу даних у одноранговому режимі.
 */
void USART1_IRQHandler(void)
{
    if (USART1->SR & USART_SR_RXNE)
    {
        /* Зчитування прийнятого символу */
        uint8_t received_char = USART1->DR;

        /* Обробка даних: збільшення значення на 1 */
        uint8_t next_val = ((received_char - '0') + 1) % 10;

        /* Очікування готовності передавача */
        while (!(USART1->SR & USART_SR_TXE));

        /* Передача результату обробки */
        USART1->DR = next_val + '0';
    }
}

int main(void)
{
    /* Увімкнення тактування GPIOA, AFIO та USART1 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN |
        RCC_APB2ENR_AFIOEN |
        RCC_APB2ENR_USART1EN;

    /* Налаштування PA9 як TX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF9;
    GPIOA->CRH |= GPIO_CRH_CNF9_1;
    GPIOA->CRH |= GPIO_CRH_MODE9;

    /* Налаштування PA10 як RX USART1 */
    GPIOA->CRH &= ~GPIO_CRH_CNF10;
    GPIOA->CRH |= GPIO_CRH_CNF10_0;
    GPIOA->CRH &= ~GPIO_CRH_MODE10;

    /* Налаштування швидкості обміну 9600 бод */
```

```
    USART1->BRR = 0x341;

    /* Формат кадру: 8 біт, без парності */
    USART1->CR1 &= ~USART_CR1_M;
    USART1->CR1 &= ~USART_CR1_PCE;

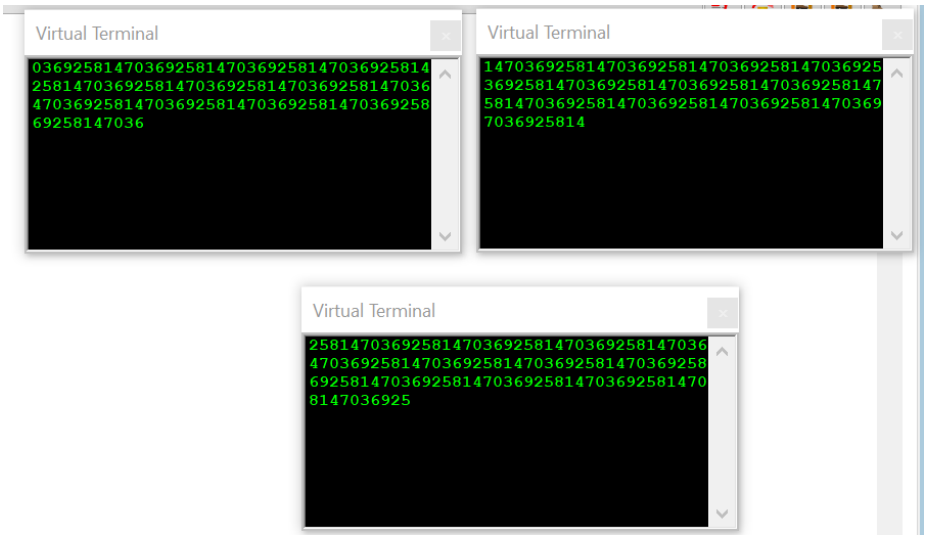
    /* Один стоп-біт */
    USART1->CR2 &= ~USART_CR2_STOP;

    /* Увімкнення USART та переривань */
    USART1->CR1 |= USART_CR1_RE |
       USART_CR1_TE |
       USART_CR1_RXNEIE |
       USART_CR1_UE;

    /* Дозвіл переривань USART1 */
    NVIC_EnableIRQ(USART1_IRQn);
    __enable_irq();

    /* Основний цикл програми */
    while (1)
    {
       /* Логіка обміну реалізована в обробнику переривань */
    }
}
```

Результати роботи системи підтверджуються відображенням символів у віртуальних терміналах та часовими діаграмами сигналів, отриманими за допомогою цифрового осцилографа в середовищі Proteus. На діаграмах спостерігається коректна послідовність передавання та приймання даних між усіма трьома мікроконтролерами, що підтверджує правильність реалізації однорангового інтерфейсу та відповідність роботи системи вимогам завдання.



Висновок.

У ході виконання лабораторної роботи було досліджено принципи організації обміну даними між мікроконтролерами STM32F103C6 з

використанням послідовного інтерфейсу UART. Було створено проєкт у середовищі Proteus, що складається з двох, а згодом і трьох мікроконтролерів, та виконано їх коректне апаратне з’єднання. Для кожного мікроконтролера реалізовано ініціалізацію апаратного інтерфейсу USART з параметрами, заданими відповідно до індивідуального варіанта.

У процесі роботи було реалізовано обмін даними у повнодуплексному режимі з використанням апаратних переривань, що дозволило забезпечити одночасну передачу та приймання інформації без використання активного опитування стану інтерфейсу. Також було перероблено програмний код для реалізації напівдуплексного режиму, у якому передавання та приймання даних відбуваються почергово. Проведене порівняння показало відмінності в алгоритмах обміну та особливості керування послідовністю операцій у різних режимах роботи інтерфейсу UART.

На завершальному етапі лабораторної роботи систему було розширено шляхом підключення третього мікроконтролера та реалізовано одноранговий обмін даними між усіма вузлами системи. Обробка, передавання та приймання даних у всіх реалізаціях здійснювалися з використанням механізму апаратних переривань, що забезпечило ефективне використання ресурсів мікроконтролерів. Результати роботи підтверджено за допомогою віртуальних терміналів та часових діаграм сигналів, отриманих у середовищі Proteus, що свідчить про правильність виконання поставлених завдань та досягнення мети лабораторної роботи.

3. Контрольні питання

3.1. Опишіть формат передавання даних протоколом UART.

Протокол UART є асинхронним послідовним протоколом передавання даних, який не використовує спільну тактову лінію, а синхронізація між передавачем і приймачем здійснюється за рахунок спеціальних бітів у структурі кадру. Формат передавання даних UART складається з кількох послідовних елементів. Передавання починається зі стартового біта, який має логічний нуль і сигналізує приймачу про початок нового кадру. Далі передаються біти даних, кількість яких зазвичай становить від п’яти до дев’яти бітів і передаються від молодшого біта до старшого. Після бітів даних може передаватися біт парності, який використовується для контролю помилок і може бути парним або непарним. Завершується кадр одним або двома стоп бітами з логічною одиницею, які забезпечують паузу між кадрами та дозволяють приймачу завершити обробку отриманих даних. Усі параметри формату кадру, зокрема швидкість передавання, кількість бітів даних, використання парності та кількість стоп бітів, мають бути однаковими для передавача і приймача.

3.2. Наведіть основні відмінності між протоколами UART і USART.

Основна відмінність між протоколами UART і USART полягає в режимах роботи та можливостях синхронізації. UART є виключно асинхронним протоколом, у якому передавання даних здійснюється без використання окремої тактової лінії, а синхронізація забезпечується стартовими та стоп бітами. USART є універсальним інтерфейсом, який може працювати як в асинхронному режимі, подібно до UART, так і в синхронному режимі з використанням додаткової тактової лінії. У синхронному режимі USART дозволяє досягти вищої швидкості та надійності передавання даних, оскільки тактовий сигнал задається передавачем. Крім того, USART зазвичай має розширений набір функцій, таких як підтримка апаратного керування потоком даних, більшу гнучкість налаштування та кращу інтеграцію з периферією мікроконтролера. Таким чином, UART є простішим інтерфейсом, тоді як USART забезпечує більшу універсальність і функціональність.

3.3. Перелічіть регістри, що використовуються для налаштування UART.

Для налаштування та керування роботою UART у мікроконтролерах використовується набір спеціалізованих регістрів. Основним регістром керування є регістр CR1, у якому задаються параметри формату кадру, увімкнення передавача і приймача, використання перевірки парності та дозволи переривань. Регістр CR2 використовується для налаштування кількості стоп бітів та деяких додаткових режимів роботи. Регістр CR3 забезпечує конфігурацію розширених можливостей, таких як керування потоком даних або спеціальні режими обміну. Швидкість передавання даних визначається за допомогою регістра BRR, який містить коефіцієнти поділу тактової частоти. Для безпосереднього передавання та приймання даних використовується регістр DR, а стан роботи інтерфейсу та наявність подій відображається у регістрі SR, який містить прапори готовності, помилок та подій переривань.

3.4. Опишіть формат передавання даних протоколом I2C.

Протокол I2C є синхронним двопровідним протоколом послідовного передавання даних, який використовує спільну лінію тактування та спільну лінію даних. Формат передавання даних I2C передбачає наявність одного ведучого пристрою та одного або кількох підлеглих. Передавання починається з умови старту, яка формується шляхом переходу лінії даних у логічний нуль при високому рівні тактової лінії. Після цього ведучий передає адресу підлеглого пристрою разом із бітом напрямку обміну. Кожен байт даних передається по вісім бітів, після чого обов’язково слідує біт підтвердження, який формується приймачем. Завершення обміну відбувається шляхом формування умови стопу, коли лінія даних переходить у логічну одиницю при високому рівні тактової лінії. Такий формат дозволяє організувати обмін даними між багатьма пристроями по одній шині з адресацією та підтвердженням кожного байта.

3.5. Перелічіть регістри, що використовуються для налаштування I2C.

Для налаштування та керування роботою інтерфейсу I2C використовується кілька основних регістрів. Регістр CR1 призначений для увімкнення інтерфейсу, генерації умов старту та стопу, а також керування основними режимами роботи. Регістр CR2 використовується для налаштування тактової частоти периферії та дозволів переривань. Адреса власного пристрою задається у регістрах OAR1 та OAR2, що дозволяє ідентифікувати підлеглий пристрій на шині. Регістр DR використовується для передавання та приймання байтів даних. Регістр SR1 містить прапори подій та помилок, таких як завершення передавання, підтвердження адреси або помилки арбітражу, а регістр SR2 використовується для зчитування додаткової інформації про стан шини та режим роботи інтерфейсу.

3.6. Опишіть формат передавання даних протоколом SPI.

Протокол SPI є синхронним повнодуплексним протоколом послідовного обміну даними, який використовує окремі лінії для передавання, приймання та синхронізації. Формат передавання даних SPI базується на використанні лінії тактового сигналу, яку генерує ведучий пристрій, лінії передавання даних від ведучого до підлеглого та лінії передавання від підлеглого до ведучого. Передавання даних відбувається пакетами фіксованої довжини, зазвичай по вісім або шістнадцять бітів, при цьому кожен тактовий імпульс відповідає передачі одного біта. Початок та завершення обміну контролюється окремою лінією вибору підлеглого, яка активується ведучим пристроєм. Дані можуть передаватися з різними фазами та полярністю тактового сигналу, що визначається режимом роботи SPI і має бути узгоджене між всіма пристроями, які беруть участь в обміні.

3.7. Перелічіть регістри, що використовуються для налаштування SPI.

Для налаштування та керування інтерфейсом SPI використовується набір керуючих і статусних регістрів. Основним є регістр CR1, у якому задається режим роботи ведучий або підлеглий, полярність і фаза тактового сигналу, швидкість передавання даних та формат кадру. Регістр CR2 використовується для керування додатковими можливостями, зокрема дозволами переривань та керуванням лінією вибору підлеглого. Регістр DR призначений для запису передаваних даних і зчитування прийнятих. Стан інтерфейсу відображається у регістрі SR, який містить прапори готовності, завершення передавання та можливих помилок. Сукупність цих регістрів забезпечує повний контроль над роботою інтерфейсу SPI та дозволяє гнучко налаштовувати обмін даними між пристроями.